

TRƯỜNG ĐẠI HỌC SƯ PHẠM HUẾ
KHOA TOÁN



DỰ ÁN NHÓM
ỨNG DỤNG SYMPY TRONG
GIẢI CÁC BÀI TOÁN LIÊN
QUAN ĐẾN GIÁ TRỊ LỚN
NHẤT VÀ GIÁ TRỊ NHỎ
NHẤT

Họ và tên: Nguyễn Thị Lệ Hà
Đào Thị Kiều Trinh
Hoàng Thị Cẩm Vân
Dương Thị Nguyên Thảo
Lớp: Toán 4A
Giảng viên: Nguyễn Đăng Minh Phúc

LỜI MỞ ĐẦU

Bài toán tìm giá trị lớn nhất (GTLN) và giá trị nhỏ nhất (GTNN) đóng vai trò cốt lõi trong tối ưu hóa toán học và thực tiễn. Khi xử lý các hàm phức tạp hay nhiều biến, việc giải thủ công thường mất nhiều thời gian và dễ phát sinh sai sót. Để giải quyết vấn đề này, thư viện SymPy của Python là một công cụ tính toán đại số mạnh mẽ, hỗ trợ tính toán ký hiệu, lấy đạo hàm và tìm cực trị tự động. Nhận thấy hiệu quả vượt trội của công nghệ trong giải toán, nhóm chúng em thực hiện dự án "Ứng dụng SymPy trong giải các bài toán liên quan đến giá trị nhỏ nhất và giá trị lớn nhất". Nhóm chúng em xin gửi lời cảm ơn chân thành nhất tới Thầy Nguyễn Đăng Minh Phúc đã tận tình định hướng và truyền đạt kiến thức chuyên môn trong suốt quá trình nhóm thực hiện dự án. Dù đã nỗ lực hoàn thành, dự án khó tránh khỏi những hạn chế. Nhóm rất mong nhận được những góp ý từ Thầy để hoàn thiện hơn.

Mục lục

1	Giới thiệu chung	4
1.1	Giới thiệu về Python	4
1.2	Giới thiệu về SymPy	4
1.3	Khái niệm giá trị lớn nhất và giá trị nhỏ nhất	5
1.4	Phương pháp tìm giá trị lớn nhất và giá trị nhỏ nhất bằng đạo hàm	5
1.5	Vai trò của SymPy trong đề tài	6
2	Các lệnh Sympy sử dụng trong bài toán giá trị lớn nhất và giá trị nhỏ nhất	8
2.1	Khai báo và sử dụng thư viện SymPy	8
2.2	Khai báo biến với lệnh <code>symbols()</code>	9
2.2.1	Cú pháp	9
2.2.2	Ví dụ	9
2.3	Tính đạo hàm với lệnh <code>diff()</code>	9
2.3.1	Cú pháp	9
2.3.2	Ví dụ	10
2.4	Giải phương trình với lệnh <code>solve()</code>	10
2.4.1	Cú pháp	10
2.4.2	Ví dụ	10
2.4.3	Sử dụng <code>Eq()</code>	11
2.5	Tìm tập nghiệm với lệnh <code>solveset()</code>	11
2.5.1	Cú pháp	11
2.5.2	Ví dụ	11
2.6	Thay giá trị vào biểu thức với lệnh <code>subs()</code>	12
2.6.1	Cú pháp	12
2.6.2	Ví dụ	12
2.7	Rút gọn biểu thức với lệnh <code>simplify()</code>	13
2.7.1	Cú pháp	13
2.7.2	Ví dụ	13

2.8	Phân tích thành nhân tử với lệnh <code>factor()</code>	13
2.8.1	Cú pháp	13
2.8.2	Ví dụ	14
2.9	Khai báo khoảng với lệnh <code>Interval()</code>	14
2.9.1	Cú pháp	14
2.10	Tìm giá trị lớn nhất với lệnh <code>maximum()</code>	15
2.10.1	Cú pháp	15
2.10.2	Ví dụ	15
2.11	Tìm giá trị nhỏ nhất với lệnh <code>minimum()</code>	16
2.11.1	Cú pháp	16
2.11.2	Ví dụ	16
2.12	Vẽ đồ thị với lệnh <code>plot()</code>	16
2.12.1	Cú pháp	17
2.12.2	Ví dụ	17
2.13	Một số lệnh SymPy đối với hàm nhiều biến	17
2.13.1	Tính đạo hàm riêng	17
2.13.2	Giải hệ phương trình tìm điểm tới hạn	18
2.14	Tóm tắt các lệnh được sử dụng	18
3	Một số bài toán tìm giá trị lớn nhất – giá trị nhỏ nhất sử dụng các lệnh của SymPy	20
3.1	Dạng 1: Tìm giá trị lớn nhất – giá trị nhỏ nhất của hàm số bậc hai trên $\mathbb{R}$	20
3.1.1	Hàm số bậc hai $y = ax^2 + bx + c$	20
3.1.2	Hàm số logarit $y = a \log_b(x) + c$	29
3.2	Dạng 2: Tìm giá trị lớn nhất – giá trị nhỏ nhất của hàm số trên một đoạn	35
3.2.1	Hàm số bậc hai	35
3.2.2	Hàm số lôgarit	44
	TÀI LIỆU THAM KHẢO	51

Chương 1

Giới thiệu chung

1.1 Giới thiệu về Python

Python là một ngôn ngữ lập trình phổ biến, có cú pháp đơn giản và dễ sử dụng. Python được ứng dụng trong nhiều lĩnh vực như phát triển phần mềm, khoa học dữ liệu, trí tuệ nhân tạo và tính toán khoa học.

Một ưu điểm của Python là có nhiều thư viện hỗ trợ tính toán. Trong lĩnh vực toán học, các thư viện giúp người dùng thực hiện các phép tính phức tạp một cách nhanh chóng và chính xác. Một trong những thư viện đó là **SymPy**, được sử dụng để tính toán toán học dưới dạng ký hiệu.

1.2 Giới thiệu về SymPy

SymPy là một thư viện mã nguồn mở của Python, được sử dụng để thực hiện các phép tính toán học ký hiệu. SymPy cho phép làm việc với các biến, biểu thức và hàm số giống như trong toán học.

Để sử dụng SymPy, ta khai báo:

```
1 import sympy as sp
```

Một số chức năng cơ bản của SymPy gồm:

- Đa thức.
- Giải tích.
- Tổ hợp.
- Ma trận.
- Hình học.
- Vẽ đồ thị hàm số.
- Xác suất thống kê.

Trong đề tài này, SymPy được sử dụng chủ yếu để hỗ trợ tính đạo hàm, giải phương trình và tìm giá trị lớn nhất, nhỏ nhất của hàm số.

1.3 Khái niệm giá trị lớn nhất và giá trị nhỏ nhất

Giá trị lớn nhất (GTLN) và giá trị nhỏ nhất (GTNN) của một hàm số $y = f(x)$ trên tập xác định hoặc tập con D là giá trị cao nhất và thấp nhất mà hàm số đó đạt được trên tập hợp đó.

- **Giá trị lớn nhất (GTLN / Max):**

- Số M được gọi là giá trị lớn nhất của hàm số $y = f(x)$ trên tập D nếu $f(x) \leq M$ với mọi $x \in D$ và tồn tại một điểm $x_0 \in D$ sao cho $f(x_0) = M$.
- Ký hiệu: $\max_D f(x) = M$.

- **Giá trị nhỏ nhất (GTNN / Min):**

- Số m được gọi là giá trị nhỏ nhất của hàm số $y = f(x)$ trên tập D nếu $f(x) \geq m$ với mọi $x \in D$ và tồn tại một điểm $x_0 \in D$ sao cho $f(x_0) = m$.
- Ký hiệu: $\min_D f(x) = m$.

1.4 Phương pháp tìm giá trị lớn nhất và giá trị nhỏ nhất bằng đạo hàm

Đối với những hàm số có thể sử dụng đạo hàm, ta thường thực hiện các dòng:

1. Xác định miền xét của hàm số.
2. Tính đạo hàm $f'(x)$.
3. Giải phương trình $f'(x) = 0$.
4. Xác định các điểm tới hạn thuộc miền xét.
5. Tính giá trị hàm số tại các điểm tới hạn và các điểm biên.
6. So sánh các giá trị để tìm GTLN và GTNN.

1.5 Vai trò của SymPy trong đề tài

Trong đề tài này, SymPy được sử dụng để hỗ trợ quá trình giải các bài toán GTLN và GTNN. Thay vì thực hiện toàn bộ phép tính bằng tay, người dùng có thể sử dụng SymPy để tính đạo hàm, giải phương trình và tính giá trị hàm số.

Việc ứng dụng SymPy giúp giảm thời gian tính toán và hạn chế một số sai sót trong quá trình thực hiện. Tuy nhiên, người sử dụng vẫn cần hiểu phương pháp toán học và kiểm tra kết quả, bởi SymPy chỉ là công cụ hỗ trợ.

Giá trị lớn nhất của hàm số bậc hai $y = ax^2 + bx + c$ ($a \neq 0$)

1. Trường hợp xét trên tập xác định $\mathbb{R}$

Hàm số bậc hai đạt giá trị lớn nhất (GTLN) trên $\mathbb{R}$ khi và chỉ khi hệ số $a < 0$.

Tọa độ đỉnh của Parabol: $I\left(-\frac{b}{2a}; -\frac{\Delta}{4a}\right)$ với $\Delta = b^2 - 4ac$.

Giá trị lớn nhất:

$$\max_{\mathbb{R}} y = -\frac{\Delta}{4a}$$

Vị trí đạt GTLN: $x = -\frac{b}{2a}$.

2. Trường hợp xét trên một đoạn $[\alpha; \beta]$

Để tìm GTLN của hàm số $y = f(x) = ax^2 + bx + c$ trên đoạn $[\alpha; \beta]$, ta thực hiện các bước sau:

1. Tính hoành độ đỉnh $x_0 = -\frac{b}{2a}$.
2. Nếu $x_0 \in [\alpha; \beta]$, ta tính các giá trị: $f(\alpha)$, $f(\beta)$ và $f(x_0)$. Khi đó:

$$\max_{[\alpha; \beta]} f(x) = \max\{f(\alpha), f(\beta), f(x_0)\}$$

3. Nếu $x_0 \notin [\alpha; \beta]$, ta chỉ tính $f(\alpha)$ và $f(\beta)$. Khi đó:

$$\max_{[\alpha; \beta]} f(x) = \max\{f(\alpha), f(\beta)\}$$

Chương 2

Các lệnh SymPy sử dụng trong bài toán giá trị lớn nhất và giá trị nhỏ nhất

Trong chương này, chúng tôi trình bày một số lệnh cơ bản của thư viện SymPy được sử dụng để giải các bài toán liên quan đến giá trị lớn nhất (GTLN) và giá trị nhỏ nhất (GTNN) của hàm số. Các lệnh được giới thiệu theo trình tự từ khai báo biến, xây dựng biểu thức, tính đạo hàm, giải phương trình đến tính giá trị hàm số và tìm GTLN, GTNN trên một miền xác định.

2.1 Khai báo và sử dụng thư viện SymPy

Để sử dụng các chức năng tính toán đại số của SymPy, trước hết cần import thư viện bằng lệnh:

```
1 import sympy as sp
```

Trong đó, `sympy` là tên của thư viện và `sp` là tên viết tắt được sử dụng trong chương trình. Việc đặt tên viết tắt giúp các câu lệnh sau ngắn gọn và thuận tiện hơn.

Ví dụ:

```
1 import sympy as sp
2 x = sp.symbols('x')
3 f = x**2 - 4*x + 3
4 print(f)
```

Kết quả:

$$x^2 - 4x + 3.$$

Lưu ý, trong SymPy, dấu `**` được sử dụng để biểu diễn phép lũy thừa. Chẳng hạn, `x**2` tương ứng với x^2 .

2.2 Khai báo biến với lệnh `symbols()`

Để SymPy thực hiện các phép tính đại số với biến, ta cần khai báo biến bằng hàm `symbols()`.

2.2.1 Cú pháp

```
1 x = sp.symbols('x')
```

Trong đó, `x` là biến ký hiệu được sử dụng trong biểu thức toán học. Có thể khai báo nhiều biến cùng lúc:

```
1 x, y = sp.symbols('x y')
```

2.2.2 Ví dụ

Xét hàm số: $f(x) = x^2 - 4x + 3$.

Ta có thể biểu diễn hàm số trong SymPy như sau:

```
1 import sympy as sp
2 x = sp.symbols('x')
3 f = x**2 - 4*x + 3
4 print(f)
```

Lệnh `symbols()` là bước đầu tiên thường được sử dụng khi xây dựng các bài toán đại số bằng SymPy.

2.3 Tính đạo hàm với lệnh `diff()`

Trong các bài toán tìm GTLN, GTNN bằng phương pháp đạo hàm, việc tính đạo hàm của hàm số là một bước quan trọng. SymPy cung cấp hàm `diff()` để thực hiện phép tính này.

2.3.1 Cú pháp

Cú pháp cơ bản:

```
1 sp.diff(bieu_thuc, bien)
```

Trong đó, `bieu_thuc` là biểu thức cần lấy đạo hàm và `bien` là biến lấy đạo hàm.

2.3.2 Ví dụ

Cho hàm số: $f(x) = x^3 - 3x^2 + 2$.

Để tính đạo hàm $f'(x)$, ta sử dụng:

```
1 import sympy as sp
2 x = sp.symbols('x')
3 f = x**3 - 3*x**2 + 2
4 f1 = sp.diff(f, x)
5 print(f1)
```

Kết quả thu được:

$$f'(x) = 3x^2 - 6x.$$

Ngoài ra, `diff()` còn có thể sử dụng để tính đạo hàm cấp cao. Chẳng hạn, đạo hàm cấp hai của $f(x)$ được tính bằng:

```
1 f2 = sp.diff(f, x, 2)
2 print(f2)
```

Kết quả:

$$f''(x) = 6x - 6.$$

2.4 Giải phương trình với lệnh `solve()`

Sau khi tính đạo hàm, để tìm các điểm tới hạn của hàm số, ta thường cần giải phương trình:

$$f'(x) = 0.$$

SymPy cung cấp hàm `solve()` để giải phương trình.

2.4.1 Cú pháp

```
1 sp.solve(phuong_trinh, bien)
```

Trong đó, `phuong_trinh` là phương trình cần giải và `bien` là biến cần tìm.

2.4.2 Ví dụ

Với hàm số:

$$f(x) = x^3 - 3x^2 + 2,$$

ta có:

$$f'(x) = 3x^2 - 6x.$$

Để tìm nghiệm của phương trình $f'(x) = 0$, ta thực hiện:

```
1 f1 = sp.diff(f, x)
2 ngiệm = sp.solve(f1, x)
3 print(ngiệm)
```

Kết quả:

$$x = 0, \quad x = 2.$$

Như vậy, các nghiệm của phương trình đạo hàm bằng 0 được SymPy trả về dưới dạng một danh sách các giá trị của x .

2.4.3 Sử dụng Eq()

Ta cũng có thể viết phương trình một cách rõ ràng bằng hàm Eq():

```
1 ngiệm = sp.solve(sp.Eq(f1, 0), x)
2 print(ngiệm)
```

Cách viết này thể hiện trực tiếp phương trình cần giải là:

$$f'(x) = 0.$$

2.5 Tìm tập nghiệm với lệnh solveset()

Bên cạnh solve(), SymPy còn cung cấp hàm solveset() để tìm tập nghiệm của phương trình.

2.5.1 Cú pháp

```
1 sp.solveset(phương_trình, biến, domain=sp.S.Reals)
```

Tham số domain=sp.S.Reals cho biết ta đang xét nghiệm thuộc tập số thực.

2.5.2 Ví dụ

Xét phương trình:

$$x^2 - 4x + 3 = 0.$$

Ta có:

```
1 x = sp.symbols('x')
2 nghiem = sp.solve(
3     x**2 - 4*x + 3,
4     x,
5     domain=sp.S.Reals
6 )
7 print(nghiem)
```

Kết quả:

$$\{1, 3\}.$$

Lệnh `solve()` đặc biệt hữu ích khi cần thể hiện nghiệm dưới dạng một tập hợp và khi muốn chỉ rõ miền nghiệm.

2.6 Thay giá trị vào biểu thức với lệnh `subs()`

Sau khi tìm được các điểm tối hạn và các điểm biên, ta cần tính giá trị của hàm số tại các điểm đó. SymPy cung cấp phương thức `subs()` để thay giá trị vào biểu thức.

2.6.1 Cú pháp

```
1 bieu_thuc.subs(bien, gia_tri)
```

2.6.2 Ví dụ

Cho:

$$f(x) = x^2 - 4x + 3.$$

Để tính $f(2)$, ta viết:

```
1 x = sp.symbols('x')
2 f = x**2 - 4*x + 3
3 gia_tri = f.subs(x, 2)
4 print(gia_tri)
```

Kết quả:

$$f(2) = -1.$$

Lệnh `subs()` rất quan trọng trong bài toán GTLN–GTNN vì sau khi tìm được các điểm cần xét, ta phải tính giá trị của hàm số tại các điểm đó.

2.7 Rút gọn biểu thức với lệnh `simplify()`

Trong quá trình tính toán, biểu thức có thể xuất hiện dưới dạng phức tạp. Hàm `simplify()` được sử dụng để rút gọn biểu thức.

2.7.1 Cú pháp

```
1 sp.simplify(bieu_thuc)
```

2.7.2 Ví dụ

Xét biểu thức:

$$\frac{x^2 - 1}{x - 1}.$$

Ta có:

```
1 x = sp.symbols('x')
2 g = (x**2 - 1)/(x - 1)
3 print(sp.simplify(g))
```

SymPy rút gọn biểu thức thành:

$$x + 1.$$

Lệnh `simplify()` giúp biểu thức dễ quan sát và thuận tiện hơn cho các bước tính toán tiếp theo.

2.8 Phân tích thành nhân tử với lệnh `factor()`

Hàm `factor()` được sử dụng để phân tích một đa thức thành tích các nhân tử.

2.8.1 Cú pháp

```
1 sp.factor(bieu_thuc)
```

2.8.2 Ví dụ

Xét:

$$f'(x) = 3x^2 - 6x.$$

Ta sử dụng:

```
1 x = sp.symbols('x')
2 f1 = 3*x**2 - 6*x
3 print(sp.factor(f1))
```

Kết quả:

$$3x(x - 2).$$

Việc phân tích thành nhân tử có thể giúp nhận biết nghiệm của phương trình $f'(x) = 0$ một cách trực quan hơn.

2.9 Khai báo khoảng với lệnh `Interval()`

Khi tìm GTLN hoặc GTNN trên một đoạn, cần xác định miền mà hàm số được xét. SymPy cung cấp đối tượng `Interval()` để biểu diễn khoảng hoặc đoạn.

2.9.1 Cú pháp

Đoạn đóng $[a, b]$ được biểu diễn bởi:

```
1 D = sp.Interval(a, b)
```

Ví dụ, đoạn $[-2, 3]$ được khai báo:

```
1 D = sp.Interval(-2, 3)
```

Để biểu diễn tập số thực, có thể sử dụng:

```
1 D = sp.S.Reals
```

Việc xác định đúng miền xét là cần thiết khi sử dụng các hàm tìm GTLN và GTNN.

2.10 Tìm giá trị lớn nhất với lệnh `maximum()`

SymPy cung cấp hàm `maximum()` để tìm giá trị lớn nhất của một hàm số trên một miền xác định.

2.10.1 Cú pháp

Trước hết cần import hàm:

```
1 from sympy.calculus.util import maximum
```

Cú pháp sử dụng:

```
1 maximum(f, x, D)
```

trong đó:

- `f`: hàm số cần xét;
- `x`: biến của hàm số;
- `D`: miền xét.

2.10.2 Ví dụ

Cho hàm số: $f(x) = -x^2 + 4x + 1$ trên đoạn $[0, 4]$.

Ta thực hiện:

```
1 import sympy as sp
2 from sympy.calculus.util import maximum
3 x = sp.symbols('x')
4 f = -x**2 + 4*x + 1
5 D = sp.Interval(0, 4)
6 M = maximum(f, x, D)
7 print(M)
```

Kết quả:

$$\max_{x \in [0, 4]} f(x) = 5.$$

Như vậy, lệnh `maximum()` cho phép xác định trực tiếp giá trị lớn nhất của hàm số trên miền đã cho.

2.11 Tìm giá trị nhỏ nhất với lệnh `minimum()`

Tương tự, SymPy cung cấp hàm `minimum()` để tìm giá trị nhỏ nhất của hàm số trên một miền xác định.

2.11.1 Cú pháp

```
1 from sympy.calculus.util import minimum
```

Sau đó sử dụng:

```
1 minimum(f, x, D)
```

2.11.2 Ví dụ

Với hàm số:

$$f(x) = -x^2 + 4x + 1$$

trên đoạn $[0, 4]$, ta có:

```
1 import sympy as sp
2 from sympy.calculus.util import minimum
3 x = sp.symbols('x')
4 f = -x**2 + 4*x + 1
5 D = sp.Interval(0, 4)
6 m = minimum(f, x, D)
7 print(m)
```

Kết quả:

$$\min_{x \in [0, 4]} f(x) = 1.$$

Hai hàm `maximum()` và `minimum()` giúp việc tìm GTLN và GTNN được thực hiện trực tiếp bằng SymPy.

2.12 Vẽ đồ thị với lệnh `plot()`

Ngoài việc thực hiện các phép tính đại số, SymPy còn hỗ trợ vẽ đồ thị hàm số. Đây là công cụ hữu ích để trực quan hóa vị trí cực trị và kiểm tra kết quả.

2.12.1 Cú pháp

```
1 sp.plot(f, (x, a, b))
```

Trong đó, (a, b) xác định khoảng giá trị của biến x được dùng để vẽ đồ thị.

2.12.2 Ví dụ

Cho hàm số: $f(x) = x^2 - 4x + 3$.

Ta sử dụng:

```
1 import sympy as sp
2 x = sp.symbols('x')
3 f = x**2 - 4*x + 3
4 sp.plot(f, (x, -2, 6))
```

Đồ thị giúp quan sát dạng của hàm số và vị trí điểm cực tiểu. Tuy nhiên, đồ thị chỉ có vai trò hỗ trợ trực quan; kết quả GTLN, GTNN trong bài toán vẫn cần được xác định bằng các phép tính toán học.

2.13 Một số lệnh SymPy đối với hàm nhiều biến

Trong các bài toán nâng cao, hàm số có thể phụ thuộc vào nhiều biến. SymPy cho phép tính đạo hàm riêng và giải hệ phương trình để tìm các điểm tới hạn.

2.13.1 Tính đạo hàm riêng

Xét hàm:

$$f(x, y) = x^2 + y^2 - 4x - 6y + 13.$$

Khai báo:

```
1 x, y = sp.symbols('x y')
2 f = x**2 + y**2 - 4*x - 6*y + 13
```

Đạo hàm riêng theo x :

```
1 fx = sp.diff(f, x)
```

Đạo hàm riêng theo y :

```
1 fy = sp.diff(f, y)
```

Kết quả:

$$\frac{\partial f}{\partial x} = 2x - 4,$$

$$\frac{\partial f}{\partial y} = 2y - 6.$$

2.13.2 Giải hệ phương trình tìm điểm tới hạn

Điểm tới hạn được xác định bằng hệ:

$$\begin{cases} \frac{\partial f}{\partial x} = 0, \\ \frac{\partial f}{\partial y} = 0. \end{cases}$$

Trong SymPy:

```
1 nghiem = sp.solve(  
2     [fx, fy],  
3     [x, y]  
4 )  
5 print(nghiem)
```

Kết quả:

$$\{x = 2, y = 3\}.$$

2.14 Tóm tắt các lệnh được sử dụng

Các lệnh quan trọng trong chương được tổng hợp trong bảng sau:

Bảng 2.1: Một số lệnh SymPy sử dụng trong bài toán GTLN–GTNN

Lệnh	Cú pháp	Chức năng
<code>symbols()</code>	<code>sp.symbols('x')</code>	Khai báo biến ký hiệu.
<code>diff()</code>	<code>sp.diff(f,x)</code>	Tính đạo hàm của hàm số.
<code>solve()</code>	<code>sp.solve(f,x)</code>	Giải phương trình hoặc hệ phương trình.
<code>solveset()</code>	<code>sp.solveset(...)</code>	Tìm tập nghiệm của phương trình.
<code>subs()</code>	<code>f.subs(x,a)</code>	Thay giá trị vào biểu thức.
<code>simplify()</code>	<code>sp.simplify(f)</code>	Rút gọn biểu thức.
<code>factor()</code>	<code>sp.factor(f)</code>	Phân tích biểu thức thành nhân tử.
<code>Interval()</code>	<code>sp.Interval(a,b)</code>	Khai báo một khoảng hoặc đoạn.
<code>maximum()</code>	<code>maximum(f,x,D)</code>	Tìm giá trị lớn nhất của hàm số trên miền D .
<code>minimum()</code>	<code>minimum(f,x,D)</code>	Tìm giá trị nhỏ nhất của hàm số trên miền D .
<code>plot()</code>	<code>sp.plot(f,(x,a,b))</code>	Vẽ đồ thị hàm số.

Như vậy, các lệnh trên cung cấp những công cụ cơ bản để xây dựng chương trình giải các bài toán GTLN–GTNN. Trong chương tiếp theo, các lệnh này sẽ được kết hợp để giải các bài toán cụ thể từ cơ bản đến nâng cao.

Chương 3

Một số bài toán tìm giá trị lớn nhất – giá trị nhỏ nhất sử dụng các lệnh của SymPy

3.1 Dạng 1: Tìm giá trị lớn nhất – giá trị nhỏ nhất của hàm số bậc hai trên $\mathbb{R}$

3.1.1 Hàm số bậc hai $y = ax^2 + bx + c$

Cho hàm số

$$\boxed{y = ax^2 + bx + c}, \quad a \neq 0.$$

Sử dụng các câu lệnh của SymPy để tìm giá trị lớn nhất hoặc giá trị nhỏ nhất của hàm số bậc hai trên $\mathbb{R}$.

Thuật toán

1. Nhập hệ số a .
2. Kiểm tra điều kiện:
 - Nếu $a = 0$, thông báo không phải hàm số bậc hai.
 - Nếu $a < 0$, hàm số có giá trị lớn nhất trên $\mathbb{R}$.
 - Nếu $a > 0$, hàm số có giá trị nhỏ nhất trên $\mathbb{R}$.

3. Nhập hai hệ số b và c .

4. Tính hoành độ đỉnh:

$$x_0 = -\frac{b}{2a}.$$

5. Tính tung độ đỉnh:

$$y_0 = ax_0^2 + bx_0 + c.$$

6. Dựa vào dấu của a để xác định giá trị lớn nhất hoặc giá trị nhỏ nhất.

7. In kết quả ra màn hình.

Chương trình Python

```
import sympy as sp

x = sp.symbols('x')

a = float(input("Nhập a: "))

if a == 0:

    print("Đây không phải hàm bậc hai.")

else:

    b = float(input("Nhập b: "))

    c = float(input("Nhập c: "))

    f = a*x**2 + b*x + c

    x0 = -b/(2*a)

    y0 = f.subs(x,x0)

    print("Hàm số:",f)

    print("Đỉnh I =", (x0,y0))

    if a > 0:

        print("Hàm số có giá trị nhỏ nhất:",y0)

        print("Đạt tại x =",x0)

    else:

        print("Hàm số có giá trị lớn nhất:",y0)

        print("Đạt tại x =",x0)
```

Giải thích từng dòng

Dòng 1: Khai báo thư viện

Lệnh SymPy

```
import sympy as sp
```

Ý nghĩa:

Nhập thư viện SymPy để hỗ trợ thực hiện các phép tính toán học, xử lý biểu thức đại số và phương trình.

Từ khóa **as sp** dùng để đặt tên viết tắt cho thư viện SymPy, giúp các câu lệnh sau ngắn gọn hơn.

Dòng 2: Khai báo biến

Lệnh khai báo biến

```
x = sp.symbols('x')
```

Ý nghĩa:

Lệnh `sp.symbols('x')` tạo một biến ký hiệu toán học có tên là x .

Biến này được sử dụng để biểu diễn hàm số dưới dạng một biểu thức đại số.

Dòng 3: Nhập hệ số a

Lệnh nhập hệ số a

```
a = float(input("Nhập a: "))
```

Ý nghĩa:

Trong câu lệnh trên:

- `input()` yêu cầu người dùng nhập giá trị của a ;
- `float()` chuyển dữ liệu nhập vào thành số thực;
- `a =` gán giá trị vừa nhập cho biến a .

Dòng 4: Kiểm tra điều kiện của a

Lệnh if

```
if a == 0:
```

Ý nghĩa:

Lệnh if dùng để kiểm tra một điều kiện.

Ở đây, $a == 0$ kiểm tra xem a có bằng 0 hay không.

Nếu điều kiện đúng, chương trình sẽ thực hiện các câu lệnh nằm trong khối if.

Dòng 5: In thông báo

Lệnh print

```
print("Đây không phải hàm bậc hai.")
```

Ý nghĩa:

Lệnh `print()` dùng để in thông báo ra màn hình khi $a = 0$.

Khi đó biểu thức $y = ax^2 + bx + c$ không còn là hàm số bậc hai.

Dòng 6: Sử dụng else

Lệnh else

```
else:
```

Ý nghĩa:

Lệnh else được thực hiện khi điều kiện của if không đúng.

Trong chương trình, nếu $a \neq 0$, chương trình tiếp tục thực hiện các câu lệnh nằm trong khối else.

Dòng 7: Nhập hệ số b

Lệnh nhập hệ số b

```
b = float(input("Nhập b: "))
```

Ý nghĩa:

Yêu cầu người dùng nhập hệ số b . Lệnh `float()` chuyển dữ liệu nhập vào thành số thực và gán cho biến b .

Dòng 8: Nhập hệ số c

Lệnh nhập hệ số c

```
c = float(input("Nhập c: "))
```

Ý nghĩa:

Yêu cầu người dùng nhập hệ số c . Lệnh `float()` chuyển dữ liệu nhập vào thành số thực và gán cho biến c .

Dòng 9: Lập hàm số

Lệnh lập hàm số

```
f = a*x**2 + b*x + c
```

Ý nghĩa:

Tạo biểu thức hàm số: $f(x) = ax^2 + bx + c$.

Dòng 10: Tính hoành độ đỉnh

Lệnh tính hoành độ đỉnh

```
x0 = -b/(2*a)
```

Ý nghĩa:

Tính hoành độ đỉnh của parabol theo công thức:

$$x_0 = -\frac{b}{2a}.$$

Biến x_0 lưu kết quả tính được.

Dòng 11: Tính tung độ đỉnh

Lệnh tính tung độ đỉnh

```
y0 = f.subs(x,x0)
```

Ý nghĩa:

Lệnh `f.subs(x,x0)` thay giá trị $x = x_0$ vào biểu thức hàm số f để tính tung độ đỉnh y_0 .

Lệnh `subs()` là viết tắt của từ “substitute”, có nghĩa là “thay thế”.

Do đó:

$$f.subs(x,x_0)$$

có nghĩa là thay x bằng x_0 trong biểu thức f .

Dòng 12: In hàm số

Lệnh in hàm số

```
print("Hàm số:",f)
```

Ý nghĩa:

Lệnh `print()` dùng để in biểu thức hàm số đã được tạo ra màn hình.

Dòng 13: In tọa độ đỉnh

Lệnh in tọa độ đỉnh

```
print("Đỉnh I =", (x0,y0))
```

Ý nghĩa:

Lệnh này in tọa độ đỉnh của parabol dưới dạng: $I(x_0; y_0)$.

Trong Python, cặp giá trị (x_0, y_0) biểu diễn tọa độ của một điểm.

Dòng 14: Kiểm tra $a > 0$

Lệnh if

```
if a > 0:
```

Ý nghĩa:

Lệnh này kiểm tra điều kiện: $a > 0$. Nếu đúng, parabol có bề lõm hướng lên và hàm số có giá trị nhỏ nhất tại đỉnh.

Dòng 15: In giá trị nhỏ nhất

Lệnh print

```
print("Hàm số có giá trị nhỏ nhất:",y0)
```

Ý nghĩa:

Nếu $a > 0$, hàm số có giá trị nhỏ nhất là:

$$y_{\min} = y_0.$$

Lệnh này dùng để in giá trị nhỏ nhất ra màn hình.

Dòng 16: In hoành độ đạt giá trị nhỏ nhất

Lệnh print

```
print("Đạt tại x =",x0)
```

Ý nghĩa:

In hoành độ x_0 tại đó hàm số đạt giá trị nhỏ nhất.

Dòng 17: Xét trường hợp $a < 0$

Lệnh else

```
else:
```

Ý nghĩa:

Nếu điều kiện $a > 0$ sai, chương trình thực hiện khối **else**.

Do trước đó đã kiểm tra $a \neq 0$, nên lúc này: $a < 0$.

Parabol có bề lõm hướng xuống và hàm số có giá trị lớn nhất tại đỉnh.

Dòng 18: In giá trị lớn nhất

Lệnh print

```
print("Hàm số có giá trị lớn nhất:",y0)
```

Ý nghĩa:

Nếu $a < 0$, hàm số có giá trị lớn nhất là:

$$y_{\max} = y_0.$$

Lệnh này dùng để in giá trị lớn nhất ra màn hình.

Dòng 19: In hoành độ đạt giá trị lớn nhất

Lệnh print

```
print("Đạt tại x =", x0)
```

Ý nghĩa:

In hoành độ x_0 tại đó hàm số đạt giá trị lớn nhất.

Bài toán minh họa

Đề bài

Viết chương trình Python sử dụng thư viện SymPy để tìm tọa độ đỉnh và giá trị nhỏ nhất của hàm số bậc hai:

$$y = 2x^2 - 4x + 1.$$

Hãy sử dụng chương trình Python để xác định tọa độ đỉnh và giá trị nhỏ nhất của hàm số.

Chương trình Python

```
import sympy as sp

x = sp.symbols('x')

a = float(input("Nhập a: "))

if a == 0:

    print("Đây không phải hàm bậc hai.")

else:

    b = float(input("Nhập b: "))

    c = float(input("Nhập c: "))

    f = a*x**2 + b*x + c
```

```

x0 = -b/(2*a)

y0 = f.subs(x,x0)

print("Hàm số:",f)

print("Đỉnh I =", (x0,y0))

if a > 0:

    print("Hàm số có giá trị nhỏ nhất:",y0)

    print("Đạt tại x =",x0)

else:

    print("Hàm số có giá trị lớn nhất:",y0)

    print("Đạt tại x =",x0)

```

Kết quả chạy chương trình

Với hàm số $y = 2x^2 - 4x + 1$, ta nhập các giá trị sau.

Kết quả hiển thị trên màn hình:

```

D:\Duancuatoi>notepad maxmin.py

D:\Duancuatoi>python maxmin.py
Nhập a: 2
Nhập b: -4
Nhập c: 1
Hàm số: 2.0*x**2 - 4.0*x + 1.0
Đỉnh I = (1.0, -1.0000000000000000)
Hàm số có giá trị nhỏ nhất: -1.0000000000000000
Đạt tại x = 1.0

```

Hình 3.1: Chạy chương trình Python

3.1.2 Hàm số logarit $y = a \log_b(x) + c$

Cho hàm số

$$\boxed{y = a \log_b(x) + c}, \quad b > 0, b \neq 1, x > 0.$$

Sử dụng các câu lệnh của SymPy để tìm giá trị lớn nhất hoặc giá trị nhỏ nhất của hàm số logarit trên tập xác định.

Thuật toán

1. Nhập các hệ số a, b, c .
2. Kiểm tra điều kiện của cơ số b :
 - Nếu $b \leq 0$ hoặc $b = 1$, thông báo cơ số không hợp lệ.
 - Nếu $b > 0$ và $b \neq 1$, tiếp tục thực hiện.
3. Xây dựng hàm số: $f(x) = a \log_b(x) + c$.
4. Kiểm tra hệ số a :
 - Nếu $a = 0$, hàm số có GTLN và GTNN bằng c .
 - Nếu $a \neq 0$, hàm số không có GTLN và GTNN trên tập xác định.
5. In kết quả ra màn hình.

Chương trình Python

```
import sympy as sp

x = sp.symbols('x', positive=True)

a = float(input("Nhập a: "))

b = float(input("Nhập cơ số b: "))

c = float(input("Nhập c: "))
```

```

if b <= 0 or b == 1:

    print("Co so b khong hop le.")

else:

    f = a*sp.log(x,b) + c

    print("Ham so:", f)

    if a == 0:

        print("GTLN = GTNN =", c)

    else:

        print("Ham so khong co GTLN va GTNN.")

```

Giải thích từng dòng

Dòng 1: Khai báo thư viện

Lệnh SymPy

```
import sympy as sp
```

Ý nghĩa:

Nhập thư viện SymPy để hỗ trợ thực hiện các phép tính toán học và xử lý biểu thức logarit.

Từ khóa `as sp` dùng để đặt tên viết tắt cho thư viện SymPy.

Dòng 2: Khai báo biến

Lệnh khai báo biến

```
x = sp.symbols('x', positive=True)
```

Ý nghĩa:

Lệnh `sp.symbols('x', positive=True)` tạo biến ký hiệu toán học x và khai báo $x > 0$.

Điều này phù hợp với tập xác định của hàm số logarit.

Dòng 3: Nhập hệ số a

Lệnh nhập hệ số a

```
a = float(input("Nhap a: "))
```

Ý nghĩa:

Trong câu lệnh trên:

- `input()` yêu cầu người dùng nhập giá trị của a ;
- `float()` chuyển dữ liệu nhập vào thành số thực;
- `a =` gán giá trị vừa nhập cho biến a .

Dòng 4: Nhập cơ số b

Lệnh nhập cơ số b

```
b = float(input("Nhap co so b: "))
```

Ý nghĩa:

Yêu cầu người dùng nhập cơ số b của hàm số logarit.

Cơ số b phải thỏa mãn điều kiện:

$$b > 0, \quad b \neq 1.$$

Dòng 5: Nhập hệ số c

Lệnh nhập hệ số c

```
c = float(input("Nhap c: "))
```

Ý nghĩa:

Yêu cầu người dùng nhập hệ số c và gán giá trị vừa nhập cho biến c .

Dòng 6: Kiểm tra điều kiện của b

Lệnh if

```
if b <= 0 or b == 1:
```

Ý nghĩa:

Lệnh `if` dùng để kiểm tra điều kiện của cơ số b .

Trong đó:

- $b \leq 0$ kiểm tra xem b có nhỏ hơn hoặc bằng 0 hay không;
- $b == 1$ kiểm tra xem b có bằng 1 hay không;
- `or` là phép hoặc.

Dòng 7: In thông báo

Lệnh `print`

```
print("Co so b khong hop le.")
```

Ý nghĩa:

Nếu điều kiện ở Dòng 6 đúng, chương trình in thông báo cơ số không hợp lệ ra màn hình.

Dòng 8: Sử dụng `else`

Lệnh `else`

```
else:
```

Ý nghĩa:

Lệnh `else` được thực hiện khi điều kiện của `if` không đúng.

Khi đó, cơ số b thỏa mãn điều kiện của hàm logarit và chương trình tiếp tục thực hiện.

Dòng 9: Lập hàm số

Lệnh lập hàm số

```
f = a*sp.log(x,b) + c
```

Ý nghĩa:

Tạo biểu thức hàm số: $f(x) = a \log_b(x) + c$.

Trong Python, `sp.log(x,b)` biểu diễn logarit cơ số b của x .

Dòng 10: In hàm số

Lệnh in hàm số

```
print("Ham so:", f)
```

Ý nghĩa:

Lệnh `print()` dùng để in biểu thức hàm số đã được tạo ra màn hình.

Dòng 11: Kiểm tra hệ số a

Lệnh if

```
if a == 0:
```

Ý nghĩa:

Lệnh này kiểm tra xem hệ số a có bằng 0 hay không.

Nếu $a = 0$, hàm số trở thành: $y = c$.

Khi đó, hàm số là hàm hằng trên tập xác định $(0; +\infty)$.

Dòng 12: In giá trị lớn nhất và nhỏ nhất

Lệnh print

```
print("GTLN = GTNN =", c)
```

Ý nghĩa:

Khi $a = 0$, hàm số có giá trị không đổi bằng c . Do đó:

$$\boxed{\text{GTLN} = \text{GTNN} = c}.$$

Dòng 13: Xét trường hợp $a \neq 0$

Lệnh else

```
else:
```

Ý nghĩa:

Nếu điều kiện $a = 0$ sai thì $a \neq 0$.

Khi đó, hàm số logarit có tập giá trị là $\mathbb{R}$, do đó hàm số không có GTLN và GTNN trên tập xác định.

Dòng 14: In kết quả

Lệnh print

```
print("Ham so khong co GTLN va GTNN.")
```

Ý nghĩa:

Lệnh này thông báo hàm số không có giá trị lớn nhất và giá trị nhỏ nhất trên tập xác định.

Bài toán minh họa

Đề bài Viết chương trình Python sử dụng thư viện SymPy để tìm GTLN và GTNN của hàm số:

$$y = 2 \log_3(x) - 1.$$

Chương trình Python

```
import sympy as sp

x = sp.symbols('x', positive=True)

a = float(input("Nhap a: "))

b = float(input("Nhap co so b: "))

c = float(input("Nhap c: "))

if b <= 0 or b == 1:

    print("Co so b khong hop le.")

else:
```

```

f = a*sp.log(x,b) + c

print("Ham so:", f)

if a == 0:

    print("GTLN = GTNN =", c)

else:

    print("Ham so khong co GTLN va GTNN.")

```

Kết quả chạy chương trình

Với hàm số $y = 2\log_3(x) - 1$, ta nhập các giá trị sau.

```

D:\Duancuatoi>notepad maxmin.py

D:\Duancuatoi>python maxmin.py
Nhap a: 2
Nhap co so b: 3
Nhap c: -1
Ham so: 1.82047845325367*log(x) - 1.0
Ham so khong co GTLN va GTNN.

```

Hình 3.2: Chạy chương trình Python

3.2 Dạng 2: Tìm giá trị lớn nhất – giá trị nhỏ nhất của hàm số trên một đoạn

3.2.1 Hàm số bậc hai

Cho hàm số:

$$y = ax^2 + bx + c, \quad a \neq 0$$

trên đoạn:

$$[m, n].$$

Sử dụng các câu lệnh của SymPy để tìm giá trị lớn nhất và giá trị nhỏ

nhất của hàm số bậc hai trên đoạn đã cho.

Thuật toán

Để tìm giá trị lớn nhất và giá trị nhỏ nhất của hàm số bậc hai trên đoạn $[m, n]$, ta thực hiện các bước sau:

1. Nhập ba hệ số a, b, c của hàm số.
2. Nhập hai đầu mút m, n của đoạn cần xét.
3. Kiểm tra điều kiện:
 - Nếu $a = 0$, thông báo không phải hàm số bậc hai.
 - Nếu $a \neq 0$, tiếp tục thực hiện.
4. Kiểm tra thứ tự của hai đầu mút:
 - Nếu $m > n$, đổi chỗ m và n .
 - Nếu $m \leq n$, giữ nguyên m, n .

Khi đó ta luôn có:

$$m \leq n.$$

5. Tính hoành độ đỉnh:

$$x_0 = -\frac{b}{2a}.$$

6. Lập danh sách các điểm cần xét gồm hai đầu mút: m, n
7. Kiểm tra xem đỉnh có thuộc đoạn $[m, n]$ hay không. Nếu:

$$m \leq x_0 \leq n,$$

thì thêm x_0 vào danh sách các điểm cần xét.

8. Tính giá trị của hàm số tại các điểm cần xét.
9. So sánh các giá trị vừa tính để xác định:

- Giá trị lớn nhất (GTLN).
- Giá trị nhỏ nhất (GTNN).

10. In kết quả ra màn hình.

Chương trình Python

```
import sympy as sp

x = sp.symbols('x')
a = float(input("Nhập a: "))
b = float(input("Nhập b: "))
c = float(input("Nhập c: "))
m = float(input("Nhập m: "))
n = float(input("Nhập n: "))

if a == 0:
    print("Đây không phải là hàm số bậc hai.")
else:
    if m > n:
        m, n = n, m

    f = a*x**2 + b*x + c
    x0 = -b/(2*a)
    diem_xet = [m, n]

    if m <= x0 <= n:
        diem_xet.append(x0)

    gia_tri = [(t, f.subs(x,t)) for t in diem_xet]
    GTLN = max(gia_tri, key=lambda p: p[1])
    GTNN = min(gia_tri, key=lambda p: p[1])
```

```
print("Hàm số:", f)
print("Hoành độ đỉnh:", x0)
print("Các điểm cần xét:", diem_xet)
print("GTLN =", GTLN[1])
print("Đạt tại x =", GTLN[0])
print("GTNN =", GTNN[1])
print("Đạt tại x =", GTNN[0])
```

Giải thích từng dòng

Dòng 1: Khai báo thư viện SymPy

Lệnh SymPy

```
import sympy as sp
```

Ý nghĩa:

Lệnh trên dùng để nhập thư viện SymPy và đặt tên viết tắt là **sp**.

Thư viện SymPy hỗ trợ thực hiện các phép tính toán học và xử lý các biểu thức đại số.

Dòng 2: Khai báo biến

Lệnh khai báo biến

```
x = sp.symbols('x')
```

Ý nghĩa:

Lệnh `sp.symbols('x')` tạo một biến ký hiệu toán học có tên là x .

Biến này được sử dụng để biểu diễn hàm số dưới dạng một biểu thức đại số.

Dòng 3: Nhập hệ số của hàm số

Lệnh nhập hệ số a, b, c

```
a = float(input("Nhập a: "))  
  
b = float(input("Nhập b: "))  
  
c = float(input("Nhập c: "))
```

Ý nghĩa:

Ba lệnh trên dùng để nhập ba hệ số a, b, c của hàm số $f(x) = ax^2 + bx + c$.

Dòng 4: Nhập hai đầu mút của đoạn

Lệnh nhập đầu mút m, n

```
m = float(input("Nhập m: "))  
  
n = float(input("Nhập n: "))
```

Ý nghĩa:

Hai lệnh trên dùng để nhập hai đầu mút của đoạn cần xét.

Đoạn cần tìm giá trị lớn nhất và giá trị nhỏ nhất là: $[m, n]$. **Dòng 5:**

Kiểm tra hàm số có phải hàm bậc hai hay không

Lệnh if

```
if a == 0:
```

Ý nghĩa:

Lệnh if dùng để kiểm tra điều kiện: $a = 0$. Nếu $a = 0$ thì biểu thức $ax^2 + bx + c$ không còn là hàm số bậc hai.

Dòng 6: Thông báo khi $a = 0$

Lệnh print

```
print("Đây không phải là hàm số bậc hai.")
```

Ý nghĩa:

Lệnh print() dùng để in thông báo ra màn hình khi $a = 0$.

Dòng 7: Sử dụng else

Lệnh else

```
else:
```

Ý nghĩa:

Lệnh `else` được thực hiện khi điều kiện `a == 0` không đúng.

Do đó, khi $a \neq 0$, chương trình tiếp tục thực hiện các câu lệnh nằm trong khối `else`.

Dòng 8: Kiểm tra thứ tự hai đầu mút

Lệnh `if`

```
if m > n:
```

Ý nghĩa:

Lệnh trên kiểm tra xem hai đầu mút có được nhập theo đúng thứ tự hay không. Nếu $m > n$, chương trình sẽ đổi chỗ hai giá trị m và n .

Dòng 9: Đổi chỗ hai đầu mút

Lệnh đổi chỗ m, n

```
m, n = n, m
```

Ý nghĩa:

Nếu người dùng nhập $m = 5$, $n = 0$, thì sau khi thực hiện lệnh trên: $m = 0$, $n = 5$. Như vậy, đoạn cần xét luôn được đưa về dạng:

$$m \leq n.$$

Dòng 10: Lập hàm số

Lệnh lập hàm số

```
f = a*x**2 + b*x + c
```

Ý nghĩa:

Lệnh trên tạo biểu thức hàm số: $f(x) = ax^2 + bx + c$.

Dòng 11: Tính hoành độ đỉnh

Lệnh tính hoành độ đỉnh

$$x_0 = -b/(2*a)$$

Ý nghĩa:

Hoành độ đỉnh của parabol được tính theo công thức:

$$x_0 = -\frac{b}{2a}.$$

Dòng 12: Lập danh sách các điểm cần xét

Lệnh lập danh sách

```
diem_xet = [m, n]
```

Ý nghĩa:

Ban đầu, chương trình xét hai đầu mút của đoạn: $x = m$, $x = n$.

Do đó danh sách ban đầu là $[m, n]$.

Dòng 13: Kiểm tra đỉnh có thuộc đoạn hay không

Lệnh if

```
if m <= x0 <= n:
```

Ý nghĩa:

Lệnh trên kiểm tra điều kiện: $m \leq x_0 \leq n$. Nếu điều kiện đúng thì đỉnh của parabol nằm trong đoạn $[m, n]$.

Dòng 14: Thêm hoành độ đỉnh vào danh sách

Lệnh append

```
diem_xet.append(x0)
```

Ý nghĩa:

Nếu $m \leq x_0 \leq n$, lệnh `append()` sẽ thêm x_0 vào danh sách các điểm cần xét.

Dòng 15: Tính giá trị hàm số tại các điểm cần xét

Lệnh tính giá trị hàm số

```
gia_tri = [(t, f.subs(x,t)) for t in diem_xet]
```

Ý nghĩa:

Lệnh trên tính giá trị của hàm số tại tất cả các điểm trong danh sách diem_xet.

Dòng 16: Tìm giá trị lớn nhất

Lệnh max

```
GTLN = max(gia_tri, key=lambda p: p[1])
```

Ý nghĩa:

Lệnh `max()` dùng để tìm phần tử có giá trị hàm số lớn nhất dựa trên thành phần thứ hai ($f(x)$).

Dòng 17: Tìm giá trị nhỏ nhất

Lệnh min

```
GTNN = min(gia_tri, key=lambda p: p[1])
```

Ý nghĩa:

Lệnh `min()` dùng để tìm phần tử có giá trị hàm số nhỏ nhất.

Dòng 18: In hàm số

Lệnh print

```
print("Hàm số:", f)
```

Ý nghĩa:

Lệnh `print()` dùng để in biểu thức hàm số đã được tạo ra màn hình.

Dòng 19: In hoành độ đỉnh

Lệnh print

```
print("Hoành độ đỉnh:", x0)
```

Ý nghĩa:

Lệnh trên in hoành độ đỉnh $x_0 = -\frac{b}{2a}$ của parabol ra màn hình.

Dòng 20: In các điểm cần xét

Lệnh print

```
print("Các điểm cần xét:", diem_xet)
```

Ý nghĩa:

Lệnh trên in danh sách các điểm được sử dụng để tìm GTLN và GTNN.

Dòng 21: In giá trị lớn nhất

Lệnh print

```
print("GTLN =", GTLN[1])
```

Ý nghĩa:

Lệnh trên in giá trị lớn nhất của hàm số trên đoạn $[m, n]$.

Dòng 22: In vị trí đạt giá trị lớn nhất

Lệnh print

```
print("Đạt tại x =", GTLN[0])
```

Ý nghĩa:

Lệnh trên in giá trị của x tại đó hàm số đạt giá trị lớn nhất.

Dòng 23: In giá trị nhỏ nhất

Lệnh print

```
print("GTNN =", GTNN[1])
```

Ý nghĩa:

Lệnh trên in giá trị nhỏ nhất của hàm số trên đoạn $[m, n]$.

Dòng 24: In vị trí đạt giá trị nhỏ nhất

Lệnh print

```
print("Đạt tại x =", GTNN[0])
```

Ý nghĩa:

Lệnh trên in giá trị của x tại đó hàm số đạt giá trị nhỏ nhất.

Bài tập minh họa

Đề bài: Tìm GTLN, GTNN của $f(x) = x^2 - 4x + 3$ trên $[0, 3]$.

Kết quả trong python:

```
D:\python>python max_min1.py
Nhập a: 1
Nhập b: -4
Nhập c: 3
Nhập m: 0
Nhập n: 3
Hàm số: 1.0*x**2 - 4.0*x + 3.0
Đoạn xét: [ 0.0 , 3.0 ]
Hoành độ đỉnh: 2.0
Các điểm cần xét: [0.0, 3.0, 2.0]
GTLN = 3.0000000000000000
Đạt tại x = 0.0
GTNN = -1.0000000000000000
Đạt tại x = 2.0
```

Hình 3.3: Chạy chương trình Python

3.2.2 Hàm số lôgarit

Cho hàm số:

$$y = \log_a(f(x)), \quad 0 < a \neq 1$$

trên đoạn:

$$[m, n].$$

Sử dụng các câu lệnh của SymPy để tìm giá trị lớn nhất và giá trị nhỏ nhất của hàm số lôgarit trên đoạn đã cho.

Thuật toán

Để tìm giá trị lớn nhất và giá trị nhỏ nhất của hàm số lôgarit $y = \log_a(f(x))$ trên đoạn $[m, n]$, ta thực hiện các bước sau:

1. Nhập cơ số a và chuỗi biểu thức $f(x)$ từ bàn phím.
2. Nhập hai đầu mút m, n của đoạn cần xét.
3. Kiểm tra thứ tự hai đầu mút:
 - Nếu $m > n$, đổi chỗ m và n .
 - Nếu $m \leq n$, giữ nguyên m, n .
4. Kiểm tra điều kiện xác định của cơ số a :
 - Nếu $a \leq 0$ hoặc $a = 1$, thông báo cơ số không hợp lệ và dừng.
 - Nếu $0 < a \neq 1$, tiếp tục thực hiện.
5. Kiểm tra điều kiện xác định của biểu thức $f(x)$:
 - Tìm giá trị nhỏ nhất của $f(x)$ trên đoạn $[m, n]$.
 - Nếu $\min_{[m, n]} f(x) \leq 0$, thông báo hàm số không xác định trên toàn bộ đoạn và dừng.
6. Tìm các điểm tới hạn của hàm số $f(x)$: $f'(x) = 0$.
7. Lập danh sách các điểm cần xét gồm hai đầu mút m, n và các điểm tới hạn thuộc đoạn $[m, n]$.
8. Tính giá trị của $y = \log_a(f(x))$ tại tất cả các điểm trong danh sách.
9. So sánh các giá trị vừa tính để xác định:
 - Giá trị lớn nhất (GTLN).
 - Giá trị nhỏ nhất (GTNN).
10. In kết quả ra màn hình.

Chương trình Python

```
import sympy as sp  
x = sp.symbols('x')
```

```

a = float(input("Nhập cơ số a :"))
f_str = input("Nhập biểu thức f(x): ")
m = float(input("Nhập m: "))
n = float(input("Nhập n: "))
if m > n:
    m, n = n, m
if a <= 0 or a == 1:
    print("Cơ số a phải thỏa mãn 0 < a != 1.")
else:
    f = sp.sympify(f_str)
    f_min = sp.calculus.util.minimum(f, x,
sp.Interval(m, n))
    if f_min <= 0:
        print("Hàm số không xác định trên đoạn [m, n].")
    else:
        y = sp.log(f) / sp.log(a)
        df = sp.diff(f, x)
        crit_pts = sp.solve(df, x)
        diem_xet = [m, n]
        for pt in crit_pts:
            if pt.is_real and m <= pt <= n:
                diem_xet.append(float(pt))
        diem_xet = list(set(diem_xet))
        gia_tri = [(t, float(y.subs(x, t))) for t in
diem_xet]
        GTLN = max(gia_tri, key=lambda p: p[1])

```

```

GTNN = min(gia_tri, key=lambda p: p[1])
print("Hàm số: y = log_", a, "(", f, ")")
print("Các điểm cần xét:", diem_xet)
print("GTLN =", round(GTLN[1], 4), "đạt tại x =",
      GTLN[0])
print("GTNN =", round(GTNN[1], 4), "đạt tại x =",
      GTNN[0])

```

Giải thích từng dòng

Dòng 1: Khai báo thư viện SymPy

Lệnh SymPy

```
import sympy as sp
```

Ý nghĩa:

Lệnh trên dùng để nhập thư viện SymPy và đặt tên viết tắt là **sp**.

Dòng 2: Khai báo biến

Lệnh khai báo biến

```
x = sp.symbols('x')
```

Ý nghĩa:

Lệnh `sp.symbols('x')` tạo một biến ký hiệu toán học có tên là x .

Dòng 3: Nhập dữ liệu đầu vào

Lệnh nhập dữ liệu

```

a = float(input("Nhập a: "))
f_str = input("Nhập f(x): ")
m = float(input("Nhập m: "))
n = float(input("Nhập n: "))

```

Ý nghĩa:

Nhập cơ số a , biểu thức $f(x)$ dạng chuỗi và hai đầu mút m, n .

Dòng 4: Kiểm tra và chuẩn hóa hai đầu mút

Lệnh if

```
if m > n: m, n = n, m
```

Ý nghĩa:

Đảm bảo đoạn cần xét luôn được đưa về dạng $m \leq n$.

Dòng 5: Kiểm tra điều kiện cơ số a

Lệnh if

```
if a <= 0 or a == 1:
```

Ý nghĩa:

Nếu cơ số không thỏa mãn $0 < a \neq 1$, chương trình dừng và thông báo lỗi.

Dòng 6: Chuyển chuỗi thành biểu thức SymPy

Lệnh sympify

```
f = sp.sympify(f_str)
```

Ý nghĩa:

Chuyển chuỗi ký tự f_str thành biểu thức đại số $f(x)$.

Dòng 7: Kiểm tra điều kiện xác định $f(x) > 0$

Lệnh minimum

```
f_min = sp.calculus.util.minimum(f, x, sp.Interval(m, n))
```

Ý nghĩa:

Tìm giá trị nhỏ nhất của $f(x)$ trên đoạn $[m, n]$. Nếu $f_{\min} \leq 0$, chương trình thông báo hàm số không xác định trên toàn bộ đoạn.

Dòng 8: Lập biểu thức lôgarit

Lệnh lôgarit

$$y = \text{sp.log}(f) / \text{sp.log}(a)$$

Ý nghĩa:

Dùng công thức đổi cơ số để tạo biểu thức:

$$y = \log_a(f(x)) = \frac{\ln(f(x))}{\ln(a)}.$$

Dòng 9: Tính đạo hàm và tìm điểm tối hạn

Lệnh đạo hàm

$$\begin{aligned} df &= \text{sp.diff}(f, x) \\ \text{crit_pts} &= \text{sp.solve}(df, x) \end{aligned}$$

Ý nghĩa:

Tính $f'(x)$ và giải phương trình $f'(x) = 0$ để tìm hoành độ các điểm tối hạn.

Dòng 10: Lập danh sách điểm cần xét

Lệnh lập danh sách

$$\text{diem_xet} = [m, n]$$

Ý nghĩa:

Thêm các điểm tối hạn là số thực và thuộc đoạn $[m, n]$ vào danh sách `diem_xet`.

Dòng 11: Tính giá trị hàm số

Lệnh tính giá trị

$$\text{gia_tri} = [(t, \text{float}(y.\text{subs}(x, t))) \text{ for } t \text{ in diem_xet}]$$

Ý nghĩa:

Thay các điểm $x = t$ vào biểu thức y để tính ra giá trị tương ứng.

Dòng 12: Tìm GTLN, GTNN

Lệnh max/min

```
GTLN = max(gia_tri, key=lambda p: p[1])  
GTNN = min(gia_tri, key=lambda p: p[1])
```

Ý nghĩa:

Dùng hàm `max()` và `min()` để tìm GTLN, GTNN.

Dòng 13: In biểu thức hàm số lôgarit

Lệnh In hàm số

```
print("Hàm số: y = log_", a, "(", f, ")")
```

Ý nghĩa:

Lệnh trên hiển thị dạng tổng quát của hàm số lôgarit ra màn hình.

Dòng 14: In danh sách các điểm cần xét

Lệnh In các điểm cần xét

```
print("Các điểm cần xét:", diem_xet)
```

Ý nghĩa:

Lệnh trên in ra danh sách các giá trị x được dùng để tính toán và so sánh.

Dòng 15: In giá trị lớn nhất

Lệnh In GTLN

```
print("GTLN =", round(GTLN[1], 4), "đạt tại x =",  
      GTLN[0])
```

Ý nghĩa:

Lệnh trên dùng để xuất giá trị lớn nhất của hàm số trên đoạn $[m, n]$ và vị trí x tương ứng.

Dòng 16: In giá trị nhỏ nhất

Lệnh In GTNN

```
print("GTNN =", round(GTNN[1], 4), "đạt tại x =",  
      GTNN[0])
```

Ý nghĩa:

Lệnh trên dùng để xuất giá trị nhỏ nhất của hàm số trên đoạn $[m, n]$ và vị trí x tương ứng.]

Bài tập minh họa

Đề bài: Tìm GTLN, GTNN của $y = \log_2(x^2 - 2x + 5)$ trên $[0, 3]$.

Kết quả trong Python:

```
D:\python>python max_min3.py  
Nhập cơ số a: 2  
Nhập biểu thức f(x): x**2-2*x+5  
Nhập m: 0  
Nhập n: 3  
Hàm số: y = log_ 2.0 ( x**2 - 2*x + 5 )  
Các điểm cần xét: [0.0, 1.0, 3.0]  
GTLN = 3.0 đạt tại x = 3.0  
GTNN = 2.0 đạt tại x = 1.0
```

Hình 3.4: Chạy chương trình Python hàm số lôgarit

Tài liệu tham khảo

- [1] Hà Huy Cử (Tổng Chủ biên), Trần Văn Tấn (Chủ biên) và các tác giả, *Toán 12 (Tập 1) - Kết nối tri thức với cuộc sống*, NXB Giáo dục Việt Nam, 2024.
- [2] Hà Huy Cử (Tổng Chủ biên), Trần Văn Tấn (Chủ biên) và các tác giả, *Toán 10 (Tập 1) - Kết nối tri thức với cuộc sống*, NXB Giáo dục Việt Nam, 2022.
- [3] Nguyễn Chí Công (Tổng Chủ biên) và các tác giả, *Tin học 10 - Kết nối tri thức với cuộc sống*, NXB Giáo dục Việt Nam, 2022.
- [4] Python Software Foundation, *Python Documentation*.